

SOF 103 Week 9: Functions

Generated by Review Agent on 2026-07-02 06:36 UTC

Source: SOF 103-Chapter 09 - Functions2024_09.ppt

Classification confidence: 80%

1. Core Knowledge Outline

中文

• 1.1 什么是函数?

- **定义:** 函数是一个可重复使用的代码块, 用于执行特定的任务。它接收输入 (参数), 处理数据, 并可能返回一个结果。
- **为什么重要:** 避免代码重复, 提高可读性, 便于调试和维护。将复杂问题分解为小的、可管理的部分 (分而治之)。
- **核心思想:** 把函数想象成一个 “黑箱” 或 “小工厂”。你给它原料 (参数), 它按照预设的工序 (函数体) 加工, 然后给你成品 (返回值)。

• 1.2 函数的组成部分

- **函数头:** 定义了函数的接口, 包括:
 - **返回类型:** 函数执行后返回什么类型的数据 (如 `int`, `double`, `void`)。
 - **函数名:** 标识函数的名称, 用于调用它。
 - **参数列表:** 函数接收的输入, 写在括号 `()` 内。每个参数都需要指定类型和名称。
- **函数体:** 用花括号 `{}` 括起来的代码块, 包含了函数执行的具体语句。
- **返回值:** 使用 `return` 关键字将结果返回给调用者。

• 1.3 函数的声明与定义

- **函数声明（原型）**：告诉编译器函数的存在、它的返回类型、名称和参数列表。通常放在文件开头或头文件中。以分号结尾。
- **函数定义**：提供函数的实际实现，即函数体。包含了函数要执行的代码。
- **1.4 函数调用**
 - 通过函数名和参数列表来执行函数。程序会跳转到被调用函数的代码，执行完毕后返回调用点继续执行。
- **1.5 参数传递**
 - **值传递**：将实参的值复制一份给形参。函数内部对形参的修改不会影响实参。
 - **引用传递**：将实参的引用（内存地址）传递给形参。函数内部对形参的修改会直接影响实参。

English

- **1.1 What is a Function?**
 - **Definition:** A reusable block of code designed to perform a specific task. It takes input (parameters), processes data, and may return a result.
 - **Why it's Important:** Avoids code duplication, improves readability, simplifies debugging and maintenance. Breaks down complex problems into smaller, manageable parts (divide and conquer).
 - **Core Idea:** Think of a function as a "black box" or a "small factory." You give it raw materials (parameters), it processes them according to a predefined procedure (function body), and then gives you a finished product (return value).
- **1.2 Components of a Function**
 - **Function Header:** Defines the function's interface, including:
 - **Return Type** : The type of data the function returns (e.g., `int` , `double` , `void`).
 - **Function Name** : An identifier used to call the function.
 - **Parameter List** : The inputs the function receives, written inside parentheses `()` . Each parameter needs a type and a name.

- **Function Body:** The block of code enclosed in curly braces `{ }` containing the statements that the function executes.
 - **Return Value:** The result sent back to the caller using the `return` keyword.
 - **1.3 Function Declaration vs. Definition**
 - **Function Declaration (Prototype):** Tells the compiler about the function's existence, its return type, name, and parameter list. Usually placed at the top of a file or in a header file. Ends with a semicolon.
 - **Function Definition:** Provides the actual implementation of the function, i.e., the function body. Contains the code the function will execute.
 - **1.4 Function Call**
 - Executing a function by using its name and providing arguments. The program jumps to the called function's code, executes it, and then returns to the point of the call to continue.
 - **1.5 Parameter Passing**
 - **Pass by Value:** A copy of the argument's value is passed to the parameter. Modifications inside the function do not affect the original argument.
 - **Pass by Reference:** A reference (memory address) to the argument is passed to the parameter. Modifications inside the function directly affect the original argument.
-
-

2. Key Concepts & Glossary

Term	Definition	Memory Anchor / Analogy
函数 (Function)	一个命名的、可重复使用的代码块，用于执行特定任务。 A named, reusable block of code that performs a specific task.	就像一台 自动售货机 。你选择商品（调用函数），投入硬币（传递参数），它就会给你想要的饮料（返回结果）。 Like a vending machine . You select an item (call the function), insert coins (pass parameters), and it gives you the drink you want (returns a result).
参数 (Parameter)	函数定义中列出的变量，用于接收调用者传递的数据。 Variables listed in the function definition that receive data passed by the caller.	就像 菜谱中的原料 。菜谱（函数）需要“鸡蛋”、“面粉”等原料（参数）才能做出蛋糕（结果）。 Like ingredients in a recipe . A recipe (function) needs "eggs", "flour", etc. (parameters) to make a cake (result).
返回值 (Return Value)	函数执行后发送回调用者的结果。 The result sent back to the caller after a function executes.	就像 自动售货机掉出的饮料 。是你调用函数后得到的结果。 Like the drink that comes out of the vending machine . It's the result you get after calling the function.
函数声明 (Function Declaration / Prototype)	告诉编译器函数的名称、返回类型和参数列表，但不包含函数体。 Tells the compiler the function's name, return type, and parameter list, but not the body.	就像 一份合同或目录 。它告诉你“有什么服务可用”以及“你需要提供什么”，但不告诉你服务是如何实现的。 Like a contract or a catalog . It tells you "what service is available" and "what you need to provide", but not how the service is implemented.
函数定义 (Function Definition)	提供函数体的完整实现。 Provides the full implementation of the function, including the body.	就像 合同背后的详细执行流程 。它包含了完成服务所需的所有具体步骤。 Like the detailed execution process behind the contract . It contains all the specific steps needed to complete the service.
值传递 (Pass by Value)	将实参的 副本 传递给函数。函数内部对参数的修改不影响原始变量。 A copy of the argument's value is	就像 你给朋友一张照片的复印件 。你的朋友可以在复印件上涂鸦，但你的原版照片完好无损。 Like giving a friend

Term	Definition	Memory Anchor / Analogy
	passed to the function. Changes inside the function do not affect the original variable.	a photocopy of a photo. Your friend can draw on the copy, but your original photo remains untouched.
引用传递 (Pass by Reference)	将实参的 内存地址 传递给函数。函数内部对参数的修改会直接影响原始变量。The memory address of the argument is passed to the function. Changes inside the function directly affect the original variable.	就像 你告诉朋友你家保险箱的密码 。你的朋友可以直接打开你的保险箱并拿走里面的东西，这会直接影响你的财产。Like telling your friend the combination to your safe . Your friend can open your safe and take things out, directly affecting your property.
<code>void</code>	一种特殊的数据类型，表示“无类型”。用于声明不返回任何值的函数。A special data type meaning "no type." Used to declare functions that do not return a value.	就像一个 只执行动作但不产生具体物品的机器 ，比如一个只负责打印“Hello”的打印机，它不返回任何纸张。Like a machine that only performs an action but doesn't produce a tangible item , like a printer that only prints "Hello" and returns nothing.

3. Critical Takeaways

中文

- 函数的核心是“封装”和“复用”**：不要重复写相同的代码。将重复的逻辑封装成一个函数，需要时调用它即可。这是编写高效、可维护代码的基石。
- 区分“声明”和“定义”**：声明是告诉编译器“有这个函数”，定义是告诉编译器“这个函数具体做什么”。声明通常放在头文件（`.h`）中，定义放在源文件（`.cpp`）中。忘记声明或定义不匹配是常见的编译错误。
- 理解“值传递”和“引用传递”的区别**：这是考试和面试中的高频考点。记住，值传递是“复制”，不影响原件；引用传递是“共享地址”，会直接影响原件。如果你希望函数修改一个外部变量，必须使用引用传递。

4. **常见误区**：初学者常犯的错误包括：函数名拼写错误、参数数量或类型不匹配、忘记写返回语句（当函数有返回类型时）、在值传递的函数内部尝试修改外部变量。

English

1. **The Core of Functions is "Encapsulation" and "Reusability"**: Don't write the same code repeatedly. Encapsulate repetitive logic into a function and call it when needed. This is the cornerstone of writing efficient and maintainable code.
 2. **Distinguish "Declaration" from "Definition"**: A declaration tells the compiler "this function exists," while a definition tells it "what this function does." Declarations are often in header files (`.h`), definitions in source files (`.cpp`). Forgetting a declaration or having a mismatch is a common compilation error.
 3. **Understand the Difference Between "Pass by Value" and "Pass by Reference"**: This is a high-frequency exam and interview topic. Remember, pass by value is a "copy" and doesn't affect the original; pass by reference is "sharing the address" and directly affects the original. If you want a function to modify an external variable, you *must* use pass by reference.
 4. **Common Misconceptions**: Beginners often make mistakes like: misspelling the function name, mismatching the number or type of arguments, forgetting to write a return statement (when the function has a return type), and trying to modify an external variable inside a function that uses pass by value.
-
-

4. Detailed Notes (AI-Expanded)

4.1 函数：程序的组织单元 / Functions: The Organizational Units of Programs

中文

想象一下，你要写一个非常复杂的程序，比如一个在线购物系统。如果所有代码都写在一个巨大的 `main` 函数里，那将会是一个噩梦：成千上万行代码混杂在一起，难以阅读、调试和维护。任何一个小改动都可能引发连锁反应，导致整个系统崩溃。

函数就是为了解决这个问题而生的。它们是程序的基本构建块，允许你将一个复杂的任务分解成一系列更小、更独立、更易于管理的子任务。每个函数专注于做好一件事。

- **为什么重要？**

- **模块化 (Modularity)**: 将程序分解成逻辑模块，每个模块由一个或多个函数实现。
- **可重用性 (Reusability)**: 一个函数可以被程序的不同部分多次调用，避免了代码重复。例如，一个 `calculateTotalPrice()` 函数可以在购物车、订单确认等多个地方使用。
- **可读性 (Readability)**: 使用有意义的函数名（如 `getUserInput()`, `validateEmail()`, `saveToDatabase()`）可以使代码像文章一样易于理解。
- **可维护性 (Maintainability)**: 如果需要修改某个功能，你只需要找到并修改对应的函数，而不用担心影响其他不相关的部分。

English

Imagine you have to write a very complex program, like an online shopping system. If all the code were written in one giant `main` function, it would be a nightmare: thousands of lines of code mixed together, hard to read, debug, and maintain. Any small change could cause a chain reaction, crashing the entire system.

Functions are designed to solve this problem. They are the fundamental building blocks of a program, allowing you to break down a complex task into a series of smaller, more independent, and manageable sub-tasks. Each function focuses on doing one thing well.

- **Why is it important?**

- **Modularity**: Decomposes the program into logical modules, each implemented by one or more functions.

- **Reusability:** A function can be called multiple times from different parts of the program, avoiding code duplication. For example, a `calculateTotalPrice()` function can be used in the shopping cart, order confirmation, and other places.
- **Readability:** Using meaningful function names (e.g., `getUserInput()`, `validateEmail()`, `saveToDatabase()`) makes the code read like an article.
- **Maintainability:** If you need to modify a feature, you only need to find and modify the corresponding function, without worrying about affecting other unrelated parts.

4.2 解剖一个函数：语法详解 / Anatomy of a Function: Syntax Explained

中文

让我们来仔细看看一个C++函数的各个部分。

```
// 函数声明 (Function Declaration) - 告诉编译器这个函数存在
int add(int a, int b);

// 函数定义 (Function Definition) - 提供函数的实际代码
int add(int a, int b) { // 函数头 (Function Header)
    int sum = a + b;    // 函数体 (Function Body) 开始
    return sum;         // 返回语句 (Return Statement)
}                      // 函数体结束
```

1. **返回类型 (Return Type):** `int`。这表示函数 `add` 在完成其工作后，会返回一个整数值给调用者。如果函数不返回任何值，则使用 `void`。
2. **函数名 (Function Name):** `add`。这是你用来调用函数的标识符。命名应该清晰、有意义。
3. **参数列表 (Parameter List):** `(int a, int b)`。这定义了函数期望接收的输入。每个参数都必须声明其类型和名称。多个参数用逗号分隔。如果函数不需要任何输入，括号内可以留空或写 `void`。
4. **函数体 (Function Body):** `{ ... }`。花括号内的代码是函数的核心逻辑。它定义了函数具体要做什么。
5. **返回语句 (Return Statement):** `return sum;`。`return` 关键字有两个作用：
 - 立即终止函数的执行。

- 将 `return` 后面的表达式的值（这里是 `sum` 的值）返回给调用者。返回值的类型必须与函数声明中的返回类型一致。

English

Let's take a closer look at the parts of a C++ function.

```
// Function Declaration - Tells the compiler this function exists
int add(int a, int b);

// Function Definition - Provides the actual code for the function
int add(int a, int b) { // Function Header
    int sum = a + b;    // Function Body starts
    return sum;         // Return Statement
}                      // Function Body ends
```

1. **Return Type:** `int`. This indicates that the `add` function will return an integer value to the caller after it finishes its work. If a function doesn't return a value, use `void`.
2. **Function Name:** `add`. This is the identifier you use to call the function. Naming should be clear and meaningful.
3. **Parameter List:** `(int a, int b)`. This defines the inputs the function expects to receive. Each parameter must declare its type and name. Multiple parameters are separated by commas. If a function takes no input, the parentheses can be left empty or contain `void`.
4. **Function Body:** `{ ... }`. The code inside the curly braces is the core logic of the function. It defines what the function specifically does.
5. **Return Statement:** `return sum;`. The `return` keyword has two purposes:
 - It immediately terminates the execution of the function.
 - It sends the value of the expression following `return` (in this case, the value of `sum`) back to the caller. The type of the returned value must match the return type declared in the function header.

4.3 函数调用：程序如何执行 / Function Call: How the Program Executes

中文

当你调用一个函数时，程序的控制权会从当前执行点转移到被调用的函数。让我们看一个例子：

```
#include <iostream>

// 函数声明
int add(int a, int b);

int main() {
    int x = 5;
    int y = 10;
    int result = add(x, y); // 函数调用 (Function Call)
    std::cout << "结果是: " << result << std::endl;
    return 0;
}

// 函数定义
int add(int a, int b) {
    int sum = a + b;
    return sum;
}
```

执行流程：

1. **main** 函数开始执行。它创建了变量 **x** (值为5) 和 **y** (值为10)。
2. 执行到 `int result = add(x, y);` 这一行。这是一个函数调用。
3. **参数传递**：程序将 **x** 和 **y** 的**值** (5和10) 复制给 **add** 函数的参数 **a** 和 **b**。此时，**a=5**，**b=10**。
4. **控制权转移**：程序暂停 **main** 函数的执行，跳转到 **add** 函数的代码。
5. **执行函数体**：**add** 函数开始执行其函数体。它计算 **a + b** 得到15，并将结果存储在 **sum** 中